# Bestellverwaltung

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;

/**
 * Java 11+ Konsolenanwendung zur Bestellverwaltung inkl. eigener JSON-
Persistenz (ohne Drittbibliotheken). * Datei: Bestellverwaltung.java */public
class Bestellverwaltung {

    // ===== Persistenz =====
    private static final Path JSON_PATH = Paths.get("bestellungen.json");

    // ===== Daten =====
    private static final List<Bestellung> bestellungen = new ArrayList<>();
    private static int nextId = 1;

    // ===== Konsole =====
    private static final Scanner SC = new Scanner(System.in);

    // ====== Datenmodell (statisch, innerhalb einer einzigen public class)
======

    public static class Bestellposition {
        public String produktname;
        public double einzelpreis;
        public int menge;

        public Bestellposition(String produktname, double einzelpreis, int
menge) {
            this.produktname = produktname;
            this.einzelpreis = einzelpreis;
            this.menge = menge;
        }

        public double gesamtpreis() {
            return einzelpreis * menge;
        }
    }

    public static class Bestellung {
        public int id;
```

```java
        public String kundenname;
        public List<Bestellposition> positionen = new ArrayList<>();

        public Bestellung(int id, String kundenname) {
            this.id = id;
            this.kundenname = kundenname;
        }

        public double gesamtpreis() {
            double sum = 0.0;
            for (Bestellposition p : positionen) sum += p.gesamtpreis();
            return sum;
        }
    }

    // ====== Einstieg ======

    public static void main(String[] args) {
        loadFromDisk();

        while (true) {
            printMenu();
            int choice = readIntInRange("Auswahl (0-6): ", 0, 6);

            switch (choice) {
                case 0:
                    println("");
                    println("Programm beendet.");
                    return;                 case 1:
                    actionNeueBestellung();
                    break;                 case 2:
                    actionAlleBestellungenAnzeigen();
                    break;                 case 3:
                    actionBestellungPerIdAnzeigen();
                    break;                 case 4:
                    actionProduktZuBestellungHinzufuegen();
                    break;                 case 5:
                    actionBestellungLoeschen();
                    break;                 case 6:
                    actionGesamtsummeAllerBestellungen();
                    break;                 default:
                    // unreachable
                    break;
            }
        }
    }
```

```java
    private static void printMenu() {
        println("");
        println("╔═══════════════════════════════════════╗");
        println("║          BESTELLVERWALTUNG            ║");
        println("╚═══════════════════════════════════════╝");
        println("╔═══════════════════════════════════════╗");
        println("║ 0) Beenden                            ║");
        println("║ 1) Neue Bestellung anlegen            ║");
        println("║ 2) Alle Bestellungen anzeigen         ║");
        println("║ 3) Bestellung per ID anzeigen         ║");
        println("║ 4) Produkt zu Bestellung hinzufügen   ║");
        println("║ 5) Bestellung löschen                 ║");
        println("║ 6) Gesamtsumme aller Bestellungen     ║");
        println("╚═══════════════════════════════════════╝");
    }

    // ====== Aktionen ======

    private static void actionNeueBestellung() {
        println("");
        String kundenname = readNonEmptyString("Kundenname: ");

        Bestellung b = new Bestellung(nextId++, kundenname);

        while (true) {
            println("");
            println("Produkt hinzufügen (leer lassen, um abzuschließen).");
            String pname = readStringAllowEmpty("Produktname: ");
            if (pname.trim().isEmpty()) break;

            double preis = readNonNegativeDouble("Einzelpreis: ");
            int menge = readPositiveInt("Menge: ");

            b.positionen.add(new Bestellposition(pname.trim(), preis, menge));
        }

        bestellungen.add(b);
        saveToDisk();
        println("");
        println("Bestellung #" + b.id + " gespeichert.");
        printBestellungDetail(b);
    }

    private static void actionAlleBestellungenAnzeigen() {
        println("");
        if (bestellungen.isEmpty()) {
```

```
            println("Keine Bestellungen vorhanden.");
            return;        }

        // Übersichtstabelle: ID | Kunde | Positionen | Summe
        int wId = 6, wKunde = 25, wPos = 11, wSum = 14;
        String top =    "┌" + repeat("─", wId)   + "┬" + repeat("─", wKunde) +
"┬" + repeat("─", wPos) + "┬" + repeat("─", wSum) + "┐";
        String mid =    "├" + repeat("─", wId)   + "┼" + repeat("─", wKunde) +
"┼" + repeat("─", wPos) + "┼" + repeat("─", wSum) + "┤";
        String bot =    "└" + repeat("─", wId)   + "┴" + repeat("─", wKunde) +
"┴" + repeat("─", wPos) + "┴" + repeat("─", wSum) + "┘";

        println(top);
        println("│" + padRight("ID", wId) + "│" + padRight("Kunde", wKunde) +
"│" + padRight("Positionen", wPos) + "│" + padRight("Summe", wSum) + "│");
        println(mid);

        double total = 0.0;
        for (Bestellung b : bestellungen) {
            total += b.gesamtpreis();
            println("│" + padRight("#" + b.id, wId)
                    + "│" + padRight(truncate(b.kundenname, wKunde), wKunde)
                    + "│" + padRight(String.valueOf(b.positionen.size()),
wPos)
                    + "│" + padRight(formatMoney(b.gesamtpreis()), wSum)
                    + "│");
        }
        println(bot);

        printSumBox("Gesamtsumme (alle Bestellungen)", total);
    }

    private static void actionBestellungPerIdAnzeigen() {
        println("");
        if (bestellungen.isEmpty()) {
            println("Keine Bestellungen vorhanden.");
            return;        }
        int id = readPositiveInt("Bestell-ID: ");
        Bestellung b = findById(id);
        if (b == null) {
            println("Keine Bestellung mit ID " + id + " gefunden.");
            return;        }
        printBestellungDetail(b);
    }

    private static void actionProduktZuBestellungHinzufuegen() {
```

```
        println("");
        if (bestellungen.isEmpty()) {
            println("Keine Bestellungen vorhanden.");
            return;        }

        int id = readPositiveInt("Bestell-ID: ");
        Bestellung b = findById(id);
        if (b == null) {
            println("Keine Bestellung mit ID " + id + " gefunden.");
            return;        }

        println("");
        String pname = readNonEmptyString("Produktname: ");
        double preis = readNonNegativeDouble("Einzelpreis: ");
        int menge = readPositiveInt("Menge: ");

        b.positionen.add(new Bestellposition(pname.trim(), preis, menge));
        saveToDisk();

        println("");
        println("Produkt hinzugefügt und gespeichert.");
        printBestellungDetail(b);
    }

    private static void actionBestellungLoeschen() {
        println("");
        if (bestellungen.isEmpty()) {
            println("Keine Bestellungen vorhanden.");
            return;        }

        int id = readPositiveInt("Bestell-ID zum Löschen: ");
        Bestellung b = findById(id);
        if (b == null) {
            println("Keine Bestellung mit ID " + id + " gefunden.");
            return;        }

        println("");
        println("Zu löschende Bestellung:");
        printBestellungDetail(b);

        String confirm = readNonEmptyString("Wirklich löschen? (j/n): 
").trim().toLowerCase(Locale.ROOT);
        if (confirm.equals("j") || confirm.equals("ja") || confirm.equals("y")
|| confirm.equals("yes")) {
            bestellungen.remove(b);
            saveToDisk();
```

```
            println("Bestellung gelöscht und gespeichert.");
        } else {
            println("Abgebrochen.");
        }
    }

    private static void actionGesamtsummeAllerBestellungen() {
        println("");
        double total = 0.0;
        for (Bestellung b : bestellungen) total += b.gesamtpreis();
        printSumBox("Gesamtsumme (alle Bestellungen)", total);
    }

    // ====== Darstellung ======

    private static void printBestellungDetail(Bestellung b) {
        println("");
        println("Bestellung #" + b.id + "  |  Kunde: " + b.kundenname);

        if (b.positionen.isEmpty()) {
            println("Keine Positionen in dieser Bestellung.");
            printSumBox("Gesamtsumme (Bestellung #" + b.id + ")", 0.0);
            return;        }

        // Format pro Position:
        // Produktname (25 Zeichen) | Menge | Einzelpreis | Gesamtpreis
int wProd = 25, wMenge = 7, wEinz = 14, wGes = 14;
        String top = "┌" + repeat("─", wProd) + "┬" + repeat("─", wMenge) +
"┬" + repeat("─", wEinz) + "┬" + repeat("─", wGes) + "┐";
        String mid = "├" + repeat("─", wProd) + "┼" + repeat("─", wMenge) +
"┼" + repeat("─", wEinz) + "┼" + repeat("─", wGes) + "┤";
        String bot = "└" + repeat("─", wProd) + "┴" + repeat("─", wMenge) +
"┴" + repeat("─", wEinz) + "┴" + repeat("─", wGes) + "┘";

        println(top);
        println("│" + padRight("Produktname", wProd)
                + "│" + padRight("Menge", wMenge)
                + "│" + padRight("Einzelpreis", wEinz)
                + "│" + padRight("Gesamtpreis", wGes)
                + "│");
        println(mid);

        for (Bestellposition p : b.positionen) {
            println("│" + padRight(truncate(p.produktname, wProd), wProd)
                    + "│" + padRight(String.valueOf(p.menge), wMenge)
                    + "│" + padRight(formatMoney(p.einzelpreis), wEinz)
```

```
                    + "│" + padRight(formatMoney(p.gesamtpreis()), wGes)
                    + "│");
        }
        println(bot);

        printSumBox("Gesamtsumme (Bestellung #" + b.id + ")",
b.gesamtpreis());
    }

    private static void printSumBox(String title, double value) {
        String t = title + ": " + formatMoney(value);
        int width = Math.max(36, t.length() + 2);

        println("");
        println("╔" + repeat("═", width) + "╗");
        println("│" + padRight(t, width) + "│");
        println("╚" + repeat("═", width) + "╝");
    }

    // ====== Eingabevalidierung ======

    private static String readNonEmptyString(String prompt) {
        while (true) {
            print(prompt);
            String s = SC.nextLine();
            if (s != null && !s.trim().isEmpty()) return s;
            println("Ungültige Eingabe: darf nicht leer sein.");
        }
    }

    private static String readStringAllowEmpty(String prompt) {
        print(prompt);
        String s = SC.nextLine();
        return (s == null) ? "" : s;
    }

    private static int readIntInRange(String prompt, int min, int max) {
        while (true) {
            print(prompt);
            String raw = SC.nextLine();
            try {
                int v = Integer.parseInt(raw.trim());
                if (v < min || v > max) {
                    println("Ungültig: Zahl muss zwischen " + min + " und " +
max + " liegen.");
                    continue;                }
```

```
                return v;
            } catch (Exception e) {
                println("Ungültige Eingabe: bitte eine ganze Zahl eingeben.");
            }
        }
    }

    private static int readPositiveInt(String prompt) {
        while (true) {
            print(prompt);
            String raw = SC.nextLine();
            try {
                int v = Integer.parseInt(raw.trim());
                if (v <= 0) {
                    println("Ungültig: muss > 0 sein.");
                    continue;                }
                return v;
            } catch (Exception e) {
                println("Ungültige Eingabe: bitte eine ganze Zahl eingeben.");
            }
        }
    }

    private static double readNonNegativeDouble(String prompt) {
        while (true) {
            print(prompt);
            String raw = SC.nextLine();
            try {
                String normalized = raw.trim().replace(',', '.');
                double v = Double.parseDouble(normalized);
                if (v < 0) {
                    println("Ungültig: darf nicht negativ sein.");
                    continue;                }
                return v;
            } catch (Exception e) {
                println("Ungültige Eingabe: bitte eine Zahl eingeben (Komma
erlaubt).");
            }
        }
    }

    // ====== Suche ======

    private static Bestellung findById(int id) {
        for (Bestellung b : bestellungen) {
            if (b.id == id) return b;
```

```
        }
        return null;
    }

    // ====== JSON Persistenz (eigener Writer/Parser) ======

    private static void loadFromDisk() {
        if (!Files.exists(JSON_PATH)) {
            // Erster Start: leer starten, kein Fehler.
            bestellungen.clear();
            nextId = 1;
            return;        }

        try {
            String content = Files.readString(JSON_PATH,
StandardCharsets.UTF_8).trim();
            if (content.isEmpty()) {
                bestellungen.clear();
                nextId = 1;
                return;            }

            // Erwartet: {"bestellungen":[{...},{...}]}
            String arr = jsonArray(content, "bestellungen");
            List<String> objs = splitTopLevelObjects(arr);

            bestellungen.clear();
            int maxId = 0;

            for (String obj : objs) {
                int id = (int) parseNumber(jsonFeld(obj, "id"));
                String kunde = jsonUnescape(stripQuotes(jsonStringFeld(obj,
"kundenname")));

                Bestellung b = new Bestellung(id, kunde);

                String posArr = jsonArray(obj, "positionen");
                List<String> posObjs = splitTopLevelObjects(posArr);

                for (String pObj : posObjs) {
                    String pname =
jsonUnescape(stripQuotes(jsonStringFeld(pObj, "produktname")));
                    double preis = parseNumber(jsonFeld(pObj, "einzelpreis"));
                    int menge = (int) parseNumber(jsonFeld(pObj, "menge"));
                    b.positionen.add(new Bestellposition(pname, preis,
menge));
                }
```

```
                bestellungen.add(b);
                if (id > maxId) maxId = id;
            }

            nextId = Math.max(1, maxId + 1);

        } catch (IOException e) {
            warn("Warnung: Konnte '" + JSON_PATH.getFileName() + "' nicht
lesen. Starte leer. (" + e.getMessage() + ")");
            bestellungen.clear();
            nextId = 1;
        } catch (Exception e) {
            warn("Warnung: JSON-Parsefehler. Starte leer. (" + e.getMessage()
+ ")");
            bestellungen.clear();
            nextId = 1;
        }
    }

    private static void saveToDisk() {
        try {
            String json = toJson();
            Files.writeString(JSON_PATH, json, StandardCharsets.UTF_8,
                    StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING, StandardOpenOption.WRITE);
        } catch (IOException e) {
            warn("Warnung: Konnte '" + JSON_PATH.getFileName() + "' nicht
speichern. (" + e.getMessage() + ")");
        }
    }

    private static String toJson() {
        StringBuilder sb = new StringBuilder();
        sb.append("{\"bestellungen\":[");
        for (int i = 0; i < bestellungen.size(); i++) {
            if (i > 0) sb.append(",");
            Bestellung b = bestellungen.get(i);
            sb.append("{");
            sb.append("\"id\":").append(b.id).append(",");

sb.append("\"kundenname\":\"").append(jsonEscape(b.kundenname)).append("\",");
            sb.append("\"positionen\":[");
            for (int j = 0; j < b.positionen.size(); j++) {
                if (j > 0) sb.append(",");
                Bestellposition p = b.positionen.get(j);
```

```
                sb.append("{");

sb.append("\"produktname\":\"").append(jsonEscape(p.produktname)).append("\","
);

sb.append("\"einzelpreis\":").append(formatNumberForJson(p.einzelpreis)).appen
d(",");
                sb.append("\"menge\":").append(p.menge);
                sb.append("}");
            }
            sb.append("]");
            sb.append("}");
        }
        sb.append("]}");
        return sb.toString();
    }

    // --- JSON Writer: manuelles Escaping
    private static String jsonEscape(String s) {
        if (s == null) return "";
        StringBuilder out = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            switch (c) {
                case '"':  out.append("\\\""); break;
                case '\\': out.append("\\\\"); break;
                case '\n': out.append("\\n");  break;
                case '\r': out.append("\\r");  break;
                case '\t': out.append("\\t");  break;
                default:   out.append(c);      break;
            }
        }
        return out.toString();
    }

    // --- JSON Parser (Zustandsautomat + geforderte Methoden)

    /**
     * Erwartet als Input ein JSON-Array-String wie "[{...},{...}]"     * und
liefert die Strings der einzelnen Objekte "{...}" auf Top-Level.     */
public static List<String> splitTopLevelObjects(String jsonArray) {
        String s = jsonArray.trim();
        if (s.isEmpty()) return Collections.emptyList();
        if (s.charAt(0) != '[') throw new IllegalArgumentException("Kein
Array: " + preview(s));
        int i = 1;
```

```java
        List<String> res = new ArrayList<>();
        boolean inString = false;
        boolean escape = false;
        int depthObj = 0;

        int startObj = -1;

        while (i < s.length()) {
            char c = s.charAt(i);

            if (inString) {
                if (escape) {
                    escape = false;
                } else if (c == '\\') {
                    escape = true;
                } else if (c == '"') {
                    inString = false;
                }
                i++;
                continue;            } else {
                if (c == '"') {
                    inString = true;
                    i++;
                    continue;                }
            }

            if (c == '{') {
                if (depthObj == 0) startObj = i;
                depthObj++;
            } else if (c == '}') {
                depthObj--;
                if (depthObj == 0 && startObj >= 0) {
                    res.add(s.substring(startObj, i + 1));
                    startObj = -1;
                }
            } else if (c == ']') {
                break;
            }

            i++;
        }

        return res;
    }
```

```
    /**
     * Liefert das String-Feld (inkl. Quotes) für einen Schlüssel, z.B.
"\"Max\"" bei kundenname.     */    public static String jsonStringFeld(String
jsonObject, String fieldName) {
        String v = findFieldValue(jsonObject, fieldName);
        v = v.trim();
        if (v.isEmpty() || v.charAt(0) != '"') {
            throw new IllegalArgumentException("Feld '" + fieldName + "' ist
kein String: " + preview(v));
        }
        return readJsonStringToken(v);
    }

    /**
     * Liefert den (nicht-String) Feldwert als Token-String (z.B. "3", "12.5",
"true", "null").     */    public static String jsonFeld(String jsonObject,
String fieldName) {
        String v = findFieldValue(jsonObject, fieldName).trim();
        if (v.isEmpty()) throw new IllegalArgumentException("Leerer Feldwert:
" + fieldName);
        if (v.charAt(0) == '"') {
            // String-Feld ist hier nicht gewünscht
            throw new IllegalArgumentException("Feld '" + fieldName + "' ist
String, erwartet Zahl/Token.");
        }
        // Token bis zum Ende/Komma/Objektende
        return readJsonValueToken(v);
    }

    /**
     * Liefert das Array (inkl. Klammern) für einen Schlüssel, z.B. "[{...}]".
*/    public static String jsonArray(String jsonObject, String fieldName) {
        String v = findFieldValue(jsonObject, fieldName).trim();
        if (v.isEmpty() || v.charAt(0) != '[') {
            throw new IllegalArgumentException("Feld '" + fieldName + "' ist
kein Array: " + preview(v));
        }
        return readJsonArrayToken(v);
    }

    /**
     * Unescapes einen JSON-String-Inhalt (ohne äußere Quotes).     */
public static String jsonUnescape(String s) {
        StringBuilder out = new StringBuilder();
        boolean esc = false;
        for (int i = 0; i < s.length(); i++) {
```

```
            char c = s.charAt(i);
            if (!esc) {
                if (c == '\\') {
                    esc = true;
                } else {
                    out.append(c);
                }
            } else {
                switch (c) {
                    case '"':  out.append('"');  break;
                    case '\\': out.append('\\'); break;
                    case 'n':  out.append('\n'); break;
                    case 'r':  out.append('\r'); break;
                    case 't':  out.append('\t'); break;
                    default:   out.append(c);    break; // tolerant
                }
                esc = false;
            }
        }
        if (esc) out.append('\\'); // tolerant
        return out.toString();
    }

    // --- Parser-Helfer (Zustandsautomat)

    private static String findFieldValue(String jsonObject, String fieldName)
{
        String s = jsonObject.trim();
        if (s.isEmpty() || s.charAt(0) != '{') throw new
IllegalArgumentException("Kein Objekt: " + preview(s));

        String needle = "\"" + fieldName + "\"";

        boolean inString = false;
        boolean escape = false;
        int depthObj = 0;
        int depthArr = 0;

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);

            if (inString) {
                if (escape) {
                    escape = false;
                } else if (c == '\\') {
                    escape = true;
```

```
                } else if (c == '"') {
                    inString = false;
                }
                continue;
            } else {
                if (c == '"') {
                    inString = true;
                }
            }

            if (!inString) {
                if (c == '{') depthObj++;
                else if (c == '}') depthObj--;
                else if (c == '[') depthArr++;
                else if (c == ']') depthArr--;
            }

            // nur auf Top-Level des Objekts (depthObj==1) und nicht in Arrays
(depthArr==0)
            if (!inString && depthObj == 1 && depthArr == 0) {
                if (matchAt(s, i, needle)) {
                    int j = i + needle.length();
                    // Skip whitespace
                    while (j < s.length() && isWs(s.charAt(j))) j++;
                    if (j >= s.length() || s.charAt(j) != ':') {
                        throw new IllegalArgumentException("Erwarte ':' nach
Feldname " + fieldName);
                    }
                    j++;
                    while (j < s.length() && isWs(s.charAt(j))) j++;
                    if (j >= s.length()) throw new
IllegalArgumentException("Kein Wert für Feld " + fieldName);
                    return s.substring(j);
                }
            }
        }

        throw new IllegalArgumentException("Feld nicht gefunden: " +
fieldName);
    }

    private static String readJsonStringToken(String s) {
        // s beginnt mit "
        boolean inString = true;
        boolean escape = false;
        int i = 1;
```

```
        while (i < s.length() && inString) {
            char c = s.charAt(i);
            if (escape) {
                escape = false;
            } else if (c == '\\') {
                escape = true;
            } else if (c == '"') {
                inString = false;
                i++;
                break;            }
            i++;
        }
        if (inString) throw new IllegalArgumentException("Unclosed string: " +
preview(s));
        return s.substring(0, i); // inklusive Quotes
    }

    private static String readJsonArrayToken(String s) {
        // s beginnt mit [
        boolean inString = false;
        boolean escape = false;
        int depth = 0;
        int i = 0;
        while (i < s.length()) {
            char c = s.charAt(i);

            if (inString) {
                if (escape) {
                    escape = false;
                } else if (c == '\\') {
                    escape = true;
                } else if (c == '"') {
                    inString = false;
                }
                i++;
                continue;            } else if (c == '"') {
                inString = true;
                i++;
                continue;            }

            if (c == '[') depth++;
            else if (c == ']') {
                depth--;
                if (depth == 0) {
                    i++;
                    return s.substring(0, i);
```

```
                }
            }
            i++;
        }
        throw new IllegalArgumentException("Unclosed array: " + preview(s));
    }

    private static String readJsonValueToken(String s) {
        // liest Token bis Komma oder Objekt/Array-Ende auf Top-Level
        boolean inString = false;
        boolean escape = false;
        int depthObj = 0;
        int depthArr = 0;

        int i = 0;
        while (i < s.length()) {
            char c = s.charAt(i);

            if (inString) {
                if (escape) {
                    escape = false;
                } else if (c == '\\') {
                    escape = true;
                } else if (c == '"') {
                    inString = false;
                }
                i++;
                continue;            } else {
                if (c == '"') {
                    inString = true;
                    i++;
                    continue;                }
            }

            if (c == '{') depthObj++;
            else if (c == '}') {
                if (depthObj == 0 && depthArr == 0) break;
                depthObj--;
            } else if (c == '[') depthArr++;
            else if (c == ']') {
                if (depthArr == 0 && depthObj == 0) break;
                depthArr--;
            }

            if (depthObj == 0 && depthArr == 0) {
                if (c == ',' || c == '}' || c == ']') break;
```

```
            }
            i++;
        }
        return s.substring(0, i).trim();
    }

    // ====== JSON Parsing Helpers (Zahlen/Strings) ======

    private static double parseNumber(String token) {
        String t = token.trim();
        if (t.isEmpty()) throw new IllegalArgumentException("Leere Zahl");
        // tolerant: JSON nutzt '.', aber wir lesen aus Datei => sollte '.'
sein
        return Double.parseDouble(t);
    }

    private static String stripQuotes(String quoted) {
        String s = quoted.trim();
        if (s.length() >= 2 && s.charAt(0) == '"' && s.charAt(s.length() - 1)
== '"') {
            return s.substring(1, s.length() - 1);
        }
        return s;
    }

    private static String formatNumberForJson(double d) {
        // JSON-konform: Punkt als Dezimaltrenner
        // Verhindert wissenschaftliche Notation im Normalfall nicht
vollständig, aber ok für typische Preise.        // Optional: via
String.format(Locale.ROOT, "%.2f", d) wäre immer 2 Nachkommastellen.
return String.format(Locale.ROOT, "%.2f", d);
    }

    // ====== String/Box Utils ======

    private static String formatMoney(double d) {
        // Anzeige: 2 Nachkommastellen, mit €-Symbol
        return String.format(Locale.GERMANY, "%.2f €", d);
    }

    private static String repeat(String s, int n) {
        if (n <= 0) return "";
        StringBuilder sb = new StringBuilder(s.length() * n);
        for (int i = 0; i < n; i++) sb.append(s);
        return sb.toString();
    }
```

```java
    private static String padRight(String s, int width) {
        if (s == null) s = "";
        if (s.length() == width) return s;
        if (s.length() > width) return s.substring(0, width);
        StringBuilder sb = new StringBuilder(width);
        sb.append(s);
        while (sb.length() < width) sb.append(' ');
        return sb.toString();
    }

    private static String truncate(String s, int width) {
        if (s == null) return "";
        if (s.length() <= width) return s;
        if (width <= 1) return s.substring(0, width);
        return s.substring(0, width - 1) + "…";
    }

    // ====== Low-level Helpers ======

    private static boolean matchAt(String s, int idx, String needle) {
        if (idx < 0 || idx + needle.length() > s.length()) return false;
        for (int i = 0; i < needle.length(); i++) {
            if (s.charAt(idx + i) != needle.charAt(i)) return false;
        }
        return true;
    }

    private static boolean isWs(char c) {
        return c == ' ' || c == '\n' || c == '\r' || c == '\t';
    }

    private static String preview(String s) {
        String t = s == null ? "" : s.replace("\n", "\\n").replace("\r",
"\\r").replace("\t", "\\t");
        if (t.length() <= 60) return t;
        return t.substring(0, 57) + "...";
    }

    private static void print(String s) {
        System.out.print(s);
    }

    private static void println(String s) {
        System.out.println(s);
    }
```

```java
    private static void warn(String s) {
        System.out.println(s);
    }
}
```